

C-Array Interoperability of MDSpan

Document #: PXXXX
Date: 2022-02-14
Project: Programming Language C++
LEWG
Reply-to: Christian Trott
<crtrott@sandia.gov>
Damien Lebrun-Grandie
<lebrungrandt@ornl.gov>
Mark Hoemmen
<mhoemmen@stellarscience.com>
K. R. Walker
<krwalker@stellarscience.com>
Daniel Sunderland
<dansunderland@gmail.com>

Contents

- [1 Revision History](#)
 - [1.1 Initial Version 2022-02 Mailing](#)
- [2 Description](#)
 - [2.1 The Issue](#)
 - [2.2 Proposal](#)
 - [2.3 Implementation Prototype](#)
- [3 Wording](#)
 - [3.1 In SubSection 22.7.X.1 \[mdspan.mdspan.overview\]](#)
 - [3.2 In SubSection 22.7.X.2 \[mdspan.mdspan.cons\]](#)

§ 1 Revision History

§ 1.1 Initial Version 2022-02 Mailing

§ 2 Description

§ 2.1 The Issue

Currently one cannot construct an `mdspan` from a multidimensional c-array.

```
int a[3][2] = {1,2,3,4,5,6};
mdspan<int, extents<2>> b(a, 3, 2); // fail
mdspan<int, extents<3, 2>> c(a); // fail
```

It only works for 1D c-arrays, which decay to pointers. However the deduction guide when using just the 1D c-array with no length argument, would deduce a rank-0 `mdspan`, which may be surprising to users:

```
int a[6] = {1,2,3,4,5,6};
mdspan b(a);
static_assert(decltype(b)::rank()==0);
```

§ 2.2 Proposal

We cannot currently fix the multidimensional c-array construction, since it is UB to alias a nested C-Array with a element type pointer - per discussion on the C++ committee reflector in January 2022. However, in practice it works (on the compilers we tested e.g. clang-based and gcc) - and it may be something the committee changes in the future - i.e. make it not-UB. We prototyped this capability, which requires an additional constructor from c-array and a few deduction guides.

What we can fix today is the deduction from 1D c-array, by adding a deduction guide from c-array constraint to rank-1 arrays.

This would enable the following:

```
{
    int a[6] = {1,2,3,4,5,6};
    mdspan b(a);
    static_assert(decltype(b)::rank()==1);
    static_assert(decltype(b)::static_extent(0)==6);
}
```

It is currently possible, and will continue to be allowed to do the following with 1D c-arrays (via the decay to pointer code path):

```
{
    int a[6] = {1,2,3,4,5,6};
    mdspan b(a, 3);
    static_assert(decltype(b)::rank()==1);
    static_assert(decltype(b)::static_extent(0)==dynamic_extent;
    // b.extent(0)==3
}
{
    int a[6] = {1,2,3,4,5,6};
    mdspan<int, extents<3>> b(a);
    static_assert(decltype(b)::rank()==1);
    static_assert(decltype(b)::static_extent(0)==3;
}
```

Adding this deduction guide now, means that if we later fix the general issue of constructing an `mdspan` from a nested c-array (which would require a core language change), we do not break prior behavior.

§ 2.3 Implementation Prototype

Based on the reference implementation of P0009 we implemented the changes in a godbolt example: [godbolt](#).

§ 3 Wording

§ 3.1 In SubSection 22.7.X.1 [`mdspan.mdspan.overview`]

— Add the following deduction guides after the class definition:

```
template<class CArray>
mdspan(CArray&)
-> mdspan<see below>;
```

```
template<class Pointer>
mdspan(Pointer&)
-> mdspan<see below>;
```

§ 3.2 In SubSection 22.7.X.2 [`mdspan.mdspan.cons`]

— Insert after paragraph 19:

```
template<class CArray>
mdspan(CArray&)
-> mdspan<see below>;
```

Constraints:

- `is_array_v<CArray>` is true.
- `rank_v<CArray>==1` is true.

Remarks:

- The deduced type is: `mdspan< std::remove_all_extents_t<CArray>, extents<extent_v<CArray,0>> >`

```
template<class Pointer>
mdspan(Pointer&)
-> mdspan<see below>;
```

Constraints:

- `is_array_v<Pointer>` is false.
- `is_pointer_v<Pointer>` is true.

Remarks:

- The deduced type is: `mdspan< std::remove_pointer_t<Pointer>, extents<> >`
- Modify paragraph 20 to be:

Constraints: `(is_convertible_v<Integrals, size_type> && ...)` is true and `sizeof...(Integrals) > 0` is true.